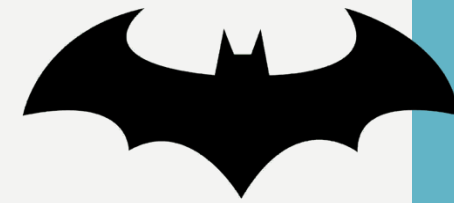




TP5 - LE MULTIVERS DE LA BATCAVE

PROTOCOLES D'AUTHENTIFICATION

CONTEXTE

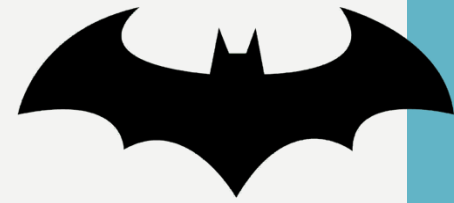


Le système de sécurité de la Batcave évolue une dernière fois ! Fini les mots de passe locaux à gérer et à sécuriser !

Votre mission est de transformer le système d'authentification actuel pour déléguer cette responsabilité à trois géants du web : **Google**, **GitHub** et **Méta** (Facebook).

L'objectif est d'offrir une expérience de connexion fluide et moderne à les utilisateurs tout en maintenant un **niveau de sécurité maximal** sur le serveur.

OBJECTIFS TECHNIQUES



Dans ce TP, vous allez concevoir l'application cliente qui va consommer **trois fournisseurs d'identité**.

Afin de maîtriser parfaitement la mécanique interne des flux réseau, **l'utilisation de librairies d'authentification**, comme [Passport.js](#), **est interdite**.

Tout doit être codé *from scratch* en utilisant uniquement Express, les requêtes HTTP natives (*fetch*) et la cryptographie standard de Node.js ([crypto](#)).



INSTRUCTIONS

CHRIS CHEVALIER

CHEVALIER@CHRIS-FREELANCE.COM

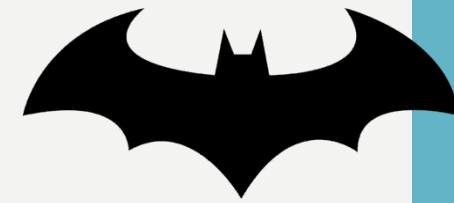
ÉTAPE 0 : ADAPTER LA BASE DE DONNÉES

Modifiez la structure de votre base de données pour accueillir ces trois fournisseurs d'identité.

 **Attention**, un identifiant `12345` sur GitHub ne correspond pas au même utilisateur que le `12345` sur Facebook !

 Assurez-vous que votre clause `ON CONFLICT` (Upsert) prenne en compte cette subtilité !

ÉTAPE 1 : CONFIGURATION



Créez une application sur les consoles respectives des trois fournisseurs :

1. [Google Cloud Console](#) ;
2. [GitHub Developer Settings](#) ;
3. [Meta for Developers](#).

Plongez dans les documentations officielles de GitHub et Facebook pour trouver :

- Les URL d'autorisation (*authorization endpoint*) ;
- Les URL d'échange de jetons (*token endpoint*) ;
- Les URL permettant de récupérer le profil utilisateur (*userinfo endpoint*) ;
- Les *scopes* exacts à demander pour récupérer au minimum l'email et le nom d'utilisateur à afficher.

ÉTAPE 2 : DÉVELOPPEMENT ASSISTÉ PAR L'IA GÉNÉRATIVE

Pour cette partie du TP, je vous encourage à utiliser l'IA générative pour comprendre et appréhender correctement les documentations officielles partagées ci-dessus.

L'objectif est d'utiliser l'outil comme un accélérateur mais avec une prise de recul **indispensable**.

Documentez, en commentaire dans votre code, au moins un cas où l'IA a montré ses limites (ex. *invention d'une URL d'API obsolète, erreur de paramètre, confusion entre OAuth 2.0 et OIDC*).



Vous devez prouver que vous restez le maître de l'architecture.

ÉTAPE 3 : INTÉGRATION SERVEUR

Implémentez le flux *Authorization Code* pour chaque fournisseur d'identité.

Intégrez la sécurité **PKCE** (*state*, *Code Verifier*, *Code Challenge*).

Que se passe-t-il si l'utilisateur clique sur "Annuler" sur la page de consentement de Facebook ou Google ?

👉 Le fournisseur va rediriger vers le callback de l'application cliente avec un paramètre d'URL spécifique (ex. `?error=access_denied`).

💡 Interceptez ce paramètre de retour et redirigez l'utilisateur vers une page d'erreur propre au lieu de laisser votre serveur crasher en cherchant un code d'autorisation inexistant.

ÉTAPE 4 : FRONTEND

Intégrez les boutons "*Se connecter avec...*" sur votre page d'accueil (`index.html`).

Avant de vous lancer dans le CSS, n'hésitez pas à faire un passage rapide sur Figma pour maquetter proprement l'alignement et la charte graphique de ces nouveaux points d'entrée afin de garantir un parcours utilisateur intuitif.



POUR ALLER PLUS LOIN

CHRIS CHEVALIER

CHEVALIER@CHRIS-FREELANCE.COM

DEVENIR SON PROPRE FOURNISSEUR D'IDENTITÉ - PHASE A

Seulement pour les étudiants ayant terminé les phases précédentes !

L'objectif ultime consiste à vous détacher des géants du web et de concevoir, *from scratch*, un écosystème d'authentification complet respectant le standard OAuth 2.0. Cette implémentation technique nécessite de faire communiquer trois entités distinctes.

Phase A : Le serveur d'autorisation (port 4000)

👉 C'est le cœur du projet. Ce serveur doit posséder sa propre base de données avec la table `users` :

- Créez une route `GET /authorize` qui affiche un formulaire de connexion HTML (email/mot de passe) ;
- Une fois l'utilisateur connecté, générez un code d'autorisation temporaire (via le module `crypto`) et stockez-le en base avec le `code_challenge` reçu ;
- Créez une route `POST /token` chargée de vérifier le code et le `code_verifier` (PKCE) puis de générer et retourner un jeton d'accès sous forme de JWT signé.

DEVENIR SON PROPRE FOURNISSEUR D'IDENTITÉ - PHASE B

Seulement pour les étudiants ayant terminé les phases précédentes !

Phase B : Le serveur de ressources (port `5000`)

👉 C'est le coffre-fort des données de la Batcave ;

- Créez une route protégée `GET /api/missions` ;
- Implémentez un middleware d'authentification qui intercepte le header `Authorization: Bearer <token>` ;
- Ce middleware doit décoder le JWT, vérifier sa signature cryptographique (avec la même clé secrète que le serveur d'autorisation) et s'assurer que sa date d'expiration (`exp`) n'est pas dépassée.

DEVENIR SON PROPRE FOURNISSEUR D'IDENTITÉ - PHASE C

Seulement pour les étudiants ayant terminé les phases précédentes !

Phase C : L'application cliente (port 3000)

👉 Adaptez le client web que vous venez de construire dans le TP principal ;

- Ajoutez un bouton "Connexion via Bat-Auth" qui initie un flux PKCE complet, mais cette fois-ci dirigé vers `http://localhost:4000/authorize` ;
- Utilisez le jeton d'accès récupéré pour requêter le port 5000 et afficher les données à l'écran.

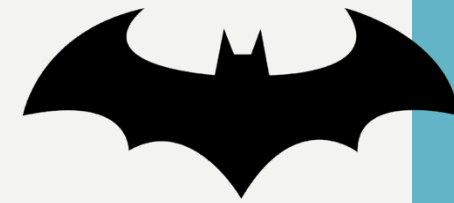


ÉVALUATION

CHRIS CHEVALIER

CHEVALIER@CHRIS-FREELANCE.COM

DEUX ÉVALUATIONS



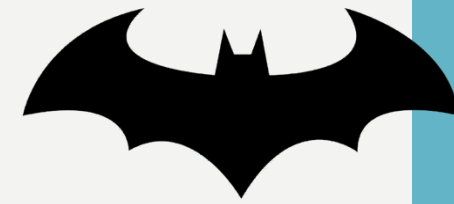
1. Livrable technique

- Dépôt Git unique organisé, idéalement, avec une branche dédiée par protocole d'authentification (du *TP1 – Basic Auth* au *TP5 – OAuth 2.0 & OIDC*) ;
- ⚠ Code freeze ce **jeudi 23 juillet à 23h59** (les commits ultérieurs ne seront pas évalués).

2. Code check individuel

- Échange oral de 10 à 20 minutes se déroulant pendant les séances de cours (clôture ce jeudi 23 juillet) ;
- Vérification de l'appropriation du code et de la maîtrise des notions théoriques vues en classe.

BARÈME D'ÉVALUATION (/20)



TP1 : Basic Auth (/2)

TP2 : Sessions et cookies (/2)

TP3 : JWT (/4)

TP4 : 2FA (/4)

TP5 : OAuth 2.0 et OpenID Connect (/5)

Transversal : Qualité, architecture et sécurité (/3)

Bonus : Section « Pour aller plus loin » (/2)

A decorative element on the left side of the slide consisting of two parallel, wavy vertical lines. The inner line is light blue and the outer line is white, creating a stylized, organic shape.

DES QUESTIONS ?

CHEVALIER@CHRIS-FREELANCE.COM